

Report Adapter Pattern

Name: Zhumasheva Akerke

Group: SE-2413

The **Adapter design pattern** is a structural pattern that allows objects with incompatible interfaces to work together. In our case, we had a class `JavaApp` with methods like `saveObject()`, `updateObject()`, `loadObject()`, and `deleteObject()`.

At the same time, we wanted to work with a unified `Database` interface, which exposes more standard methods: `insert()`, `update()`, `select()`, and `remove()`.

The adapter pattern helps to "translate" calls from the `Database` interface into the corresponding methods of `JavaApp`.

Code Example

1. Interface – Database This interface defines the operations that the client expects:

```
public interface Database { 2 usages 1 implementation
    public void insert(); 1 usage 1 implementation

    public void update(); 1 usage 1 implementation

    public void select(); 1 usage 1 implementation

    public void remove(); 1 usage 1 implementation
    default void search(String query) { no usages
        System.out.println("Searching in database for: " + query);
    }

    default void clear() { no usages
        System.out.println("Clearing all records from database...");
    }
}
```

Here, also added default methods (search and clear) to provide common functionality directly in the interface.

2. Adapter – JavaApp This is the existing class with its own method names:

```
public class JavaApp { no usages 1 inheritor
    public void saveObject(){ no usages
        System.out.println("Saving Object");
    }

    public void updateObject(){ no usages
        System.out.println("Updating Object");
    }

    public void loadObject(){ no usages
        System.out.println("Loading Object");
    }

    public void deleteObject(){ no usages
        System.out.println("Deleting Object");
    }
}
```

3. Adapter – Adapter The adapter connects with both

```
public class Adapter extends JavaApp implements Database{ no usage
    @Override 1 usage
    public void insert() {
        saveObject();
    }

    @Override 1 usage
    public void update() {
        updateObject();
    }

    @Override 1 usage
    public void select() {
        loadObject();
    }

    @Override 1 usage
    public void remove() {
        deleteObject();
    }
}
```

The adapter inherits from JavaApp and implements the Database interface. Each Database method delegates its work to the corresponding JavaApp method.

4. Client – Client

```
public class Client {  
    public static void main(String[] args) {  
        Database database = new Adapter();  
  
        database.insert();  
        database.update();  
        database.select();  
        database.remove();  
        database.search( query: "id=101");  
        database.clear();  
    }  
}
```

Conclusion By implementing the Adapter pattern, we successfully connected two incompatible interfaces:

Database – the expected standard interface,

JavaApp – the existing class with different method names.

The adapter (Adapter) acts as a bridge between them, allowing the client to work with Database while internally reusing the functionality of JavaApp.

This shows a clear understanding of the Adapter pattern and its role in real-world development.

Link:<https://github.com/xzxz8741/Adapter>